

1. Introducción

El presente trabajo documenta la investigación comparativa y la implementación práctica de herramientas de automatización de pruebas y de análisis estático de código, aplicadas al proyecto integrador del equipo: **Cal.com**, una plataforma open-source para la programación de citas y gestión de disponibilidad, construida sobre Next.js, TypeScript y Prisma, con PostgreSQL como motor de base de datos. El repositorio oficial sobre el cual se trabaja se encuentra disponible en <https://github.com/UP240080/EyMDSW-Calcom>.

A lo largo del cuatrimestre el equipo ha analizado tres funcionalidades críticas de la plataforma –gestión de disponibilidad, flujo de reserva de citas y panel administrativo– bajo los atributos de calidad de usabilidad, compatibilidad y accesibilidad. La presente tarea complementa ese análisis al incorporar dos frentes adicionales: por un lado, una investigación sobre el estado del arte de los frameworks de pruebas end-to-end (E2E) y de las herramientas de análisis estático de código; por otro lado, la aplicación práctica de Selenium con pytest y de SonarQube directamente sobre el código del proyecto.

Se investigarán al menos tres frameworks de automatización E2E (Selenium, Cypress y Playwright) y al menos tres herramientas de análisis estático (SonarQube, ESLint y Semgrep), comparando sus características técnicas para justificar su pertinencia frente al stack tecnológico de Cal.com. Posteriormente, el equipo implementará una suite de pruebas E2E con Page Object Model sobre un módulo real del flujo de reserva, y ejecutará un análisis estático con SonarQube para identificar issues de calidad, los cuales serán priorizados en un plan de remediación de deuda técnica. Finalmente, se documentará la integración de ambas prácticas en un pipeline de integración continua.

2. Desarrollo

Esta sección integra los seis componentes solicitados por la tarea: investigación de frameworks E2E, investigación de herramientas de análisis estático, implementación de pruebas con Selenium, análisis estático con SonarQube, integración en CI y un plan de remediación de deuda técnica. Las dos primeras subsecciones (investigación) sientan las bases conceptuales que justifican las decisiones tomadas en las subsecciones prácticas que les siguen.

2.1. Investigación: frameworks de automatización E2E

Las pruebas end-to-end (E2E) validan el comportamiento de una aplicación desde la perspectiva del usuario final, simulando interacciones reales sobre la interfaz en un navegador. Jorgensen (2022) ubica este tipo de pruebas en el nivel más alto de la pirámide de pruebas, ya que ejercitan el sistema integrado de punta a punta, a diferencia de las pruebas unitarias que aíslan componentes individuales. Para el caso de Cal.com, cuyo flujo de reserva de citas involucra múltiples pasos secuenciales (selección de tipo de evento, elección de fecha y hora, captura de datos y confirmación), este tipo de prueba resulta indispensable para validar la experiencia completa del usuario.

Se compararon tres de los frameworks más utilizados actualmente en la industria: **Selenium**, **Cypress** y **Playwright**.

Selenium es el framework más longevo de los tres y opera mediante el protocolo WebDriver, lo que le permite comunicarse con prácticamente cualquier navegador a través de su respectivo driver (García, 2022). Su madurez se traduce en una comunidad extensa y en soporte para múltiples lenguajes de programación, pero también en una curva de aprendizaje más pronunciada, ya que el manejo de esperas y la gestión de drivers recae casi por completo en quien escribe la prueba (García, 2022).

Cypress, en cambio, se ejecuta dentro del mismo bucle de eventos del navegador, lo que le da acceso directo al DOM y a las peticiones de red de la aplicación bajo prueba. Esta arquitectura simplifica considerablemente el manejo de esperas, ya que Cypress reintenta automáticamente las aserciones hasta que se cumplen o se agota un tiempo límite (Aniche, 2022). Su principal limitación es que está acotado al ecosistema JavaScript/TypeScript y, históricamente, a navegadores basados en Chromium, Firefox y Edge.

Playwright, desarrollado por Microsoft, combina varias de las ventajas de los dos anteriores: ofrece soporte multinavegador real (Chromium, Firefox y WebKit) desde un único motor de automatización, esperas automáticas integradas (auto-waiting) y soporte para múltiples lenguajes, incluyendo TypeScript, Python, Java y C. Su arquitectura basada en contextos de navegador aislados le permite ejecutar pruebas en paralelo con mayor veloci-

dad que las arquitecturas tradicionales basadas en WebDriver (Khorikov, 2020, respecto a la importancia de la velocidad de ejecución para mantener suites de prueba sostenibles).

Criterio	Selenium	Cypress	Playwright
Lenguajes soportados	Java, Python, C#, JS/TS, Ruby, Kotlin	JavaScript / TypeScript	JS/TS, Python, Java, C#
Curva de aprendizaje	Alta (gestión manual de drivers y esperas)	Media (API declarativa, propia de JS)	Media-baja (auto-wait, API moderna)
Velocidad de ejecución	Media (overhead del protocolo WebDriver)	Alta (ejecución dentro del navegador)	Alta (contextos aislados, ejecución paralela nativa)
Manejo de esperas	Manual (<code>WebDriverWait</code>)	Automático (retry-ability integrado)	Automático (auto-waiting integrado)
Soporte de navegadores	Chrome, Firefox, Edge, Safari (vía drivers)	Chrome, Edge, Firefox (Safari experimental)	Chromium, Firefox y WebKit nativos
Comunidad	Muy amplia, desde 2004	Amplia, en crecimiento desde 2017	En crecimiento acelerado, respaldo de Microsoft
Licencia	Apache 2.0 (open source)	MIT (núcleo open source; Cypress Cloud es de pago)	Apache 2.0 (open source)

Cuadro 1: Comparativa de frameworks de automatización E2E. Fuente: elaboración propia con base en García (2022), Aniche (2022) y documentación oficial (selenium.dev, docs.cypress.io, playwright.dev).

Recomendación. Dado que Cal.com es una aplicación construida íntegramente en TypeScript sobre Next.js, Playwright sería la opción más adecuada para un entorno de producción: ofrece soporte nativo del lenguaje del proyecto, cobertura multinavegador real y una velocidad de ejecución superior, factores relevantes para una aplicación con flujos críticos como la reserva de citas. Sin embargo, y tal como lo permite el lineamiento de la tarea, la implementación práctica de este trabajo (sección 4.3) se realiza con **Selenium** + **pytest**, por ser la herramienta trabajada en las sesiones de clase y por contar con un ecosistema robusto de documentación y patrones (como Page Object Model) ampliamente documentados por García (2022). Esta decisión no contradice la recomendación anterior, sino que distingue entre la herramienta técnicamente más idónea para el stack del proyecto y la herramienta pedagógicamente adecuada para los objetivos de esta unidad.

2.2. Investigación: análisis estático de código

El análisis estático de código examina el código fuente sin ejecutarlo, identificando patrones que indican posibles defectos, vulnerabilidades de seguridad, código duplicado o violaciones a buenas prácticas de diseño (Khorikov, 2020). Esto lo distingue del análisis dinámico –como las pruebas E2E descritas en la sección anterior– que observa el comportamiento real del sistema durante su ejecución y, por lo tanto, solo puede detectar defectos en las rutas de código que efectivamente se ejercitan durante la prueba. El análisis estático, en cambio, recorre el cien por ciento del código fuente, lo que le permite detectar problemas de forma temprana, antes de que el sistema esté siquiera en ejecución, aunque con el costo de generar en ocasiones falsos positivos.

Se compararon tres herramientas de análisis estático relevantes para el stack de Cal.com: **SonarQube**, **ESLint** y **Semgrep**.

SonarQube es una plataforma integral que evalúa cinco dimensiones de calidad (confiabilidad, seguridad, mantenibilidad, cobertura y duplicación) y soporta de forma nativa más de 30 lenguajes, incluyendo TypeScript y JavaScript. Su fortaleza principal es ofrecer

una visión consolidada del estado de calidad de todo un repositorio, junto con un mecanismo de *Quality Gate* que puede bloquear integraciones que no cumplan los umbrales definidos.

ESLint, por su parte, es un linter especializado únicamente en JavaScript y TypeScript, enfocado en detectar errores de sintaxis, problemas de estilo y antipatrones específicos del lenguaje en tiempo de escritura, generalmente integrado directamente en el editor o como parte de los hooks de pre-commit. A diferencia de SonarQube, no ofrece una visión histórica ni un tablero centralizado de calidad, pero su retroalimentación es prácticamente inmediata para quien programa.

Semgrep se distingue por su enfoque basado en reglas semánticas ligeras, definidas en patrones legibles similares al propio código, lo que facilita escribir reglas personalizadas para detectar vulnerabilidades de seguridad específicas (por ejemplo, uso indebido de consultas SQL sin parametrizar) de forma más ágil que con motores de análisis más pesados (Aniche, 2022, respecto a la importancia de detectar errores comunes de forma temprana y automatizada).

Criterio	SonarQube	ESLint	Semgrep
Tipo de defectos detectados	Bugs, vulnerabilidades, code smells, duplicación, hotspots de seguridad	Errores de sintaxis, antipatrones de JS/TS, estilo de código	Vulnerabilidades de seguridad y patrones personalizados
Lenguajes soportados	+30 lenguajes (incluye TS/JS)	JavaScript / TypeScript (vía plugins)	+20 lenguajes, reglas personalizables
Integración en flujo de trabajo	Servidor centralizado / CI, Quality Gate	Editor / pre-commit / CI	CLI / CI, reglas como código
Visión histórica del proyecto	Sí (tablero con tendencias)	No	Limitada
Licencia	Community Edition gratuita; ediciones avanzadas de pago	MIT (open source)	Open source (núcleo); Semgrep Cloud de pago

Cuadro 2: Comparativa de herramientas de análisis estático de código. Fuente: elaboración propia con base en Khorikov (2020), Aniche (2022) y documentación oficial (docs.sonarsource.com, eslint.org, semgrep.dev).

Justificación para el stack de Cal.com. SonarQube resulta adecuado para este proyecto porque analiza de forma nativa TypeScript y JavaScript –los lenguajes en los que está escrito prácticamente todo el código de Cal.com–, y porque su mecanismo de *Quality Gate* permite establecer un criterio objetivo y automatizable sobre el cual el equipo puede decidir si un cambio es apto para integrarse al proyecto, algo especialmente valioso dado el volumen y la complejidad del repositorio descritos como riesgo en la elección del caso de estudio. Además, SonarQube puede consumir directamente el archivo `coverage.xml` generado por `pytest-cov`, lo que permite vincular en un mismo tablero la cobertura producida por la suite de pruebas E2E (sección 4.3) con el resto de las métricas estáticas del código (sección 4.4). Por estas razones, se selecciona SonarQube como herramienta principal de análisis estático para la práctica, complementando su uso, de ser necesario, con las reglas de ESLint ya presentes en la configuración original del proyecto Cal.com.